

APP DE TRADING AUTOMÁTICO CON MODELOS COMPUTACIONALES

Autor

Alejandro López López

Tutores

Jose Tomás Palma Mendez

Institución

Universidad de Murcia

Facultad de Informática

Grado en Ingeniería Informática

Fecha de entrega: 18 de febrero de 2026

Resumen

Este Trabajo de Fin de Grado presenta el desarrollo de una plataforma de trading algorítmico orientada a estudiantes. El objetivo principal es proporcionar un entorno controlado y seguro donde los usuarios puedan experimentar con estrategias de inversión utilizando datos reales del mercado de criptomonedas, sin exponer capital real al riesgo financiero.

La arquitectura del sistema es híbrida y modular, integrando un backend robusto en Java (Spring Boot) para la gestión contable y el ciclo de vida de las operaciones, con un motor de cálculo en Python. Esta dualidad permite ejecutar simulaciones precisas (Backtesting) y operativa en tiempo real simulada (Paper Trading). Una característica de la plataforma es que tanto por backtesting como por paper trading, pueden ver los resultados y las operaciones realizadas. También es posible agrupar los datos históricos de varias monedas para luego entrenar modelos de Inteligencia Artificial, ya sea con indicadores básicos o con los resultados de las estrategias.

Palabras clave: Trading Educativo, Criptomonedas, Spring Boot, Python, Inteligencia Artificial, Backtesting.

Abstract

This Final Degree Project presents the development of an algorithmic trading platform tailored for students. The primary objective is to provide a controlled and secure environment where users can experiment with investment strategies using real-world cryptocurrency market data, without exposing actual capital to financial risk.

The system architecture is hybrid and modular, integrating a robust Java (Spring Boot) backend for accounting management and operational lifecycle, with a Python calculation engine. This duality enables the execution of accurate simulations (Backtesting) and simulated real-time trading (Paper Trading). A key feature of the platform is that it provides detailed visibility into the results and operations performed in both modes. Furthermore, the system allows for the aggregation of historical data from multiple cryptocurrencies to train Artificial Intelligence models, utilizing either basic technical indicators or the specific results generated by the strategies.

Keywords: Educational Trading, Cryptocurrencies, Spring Boot, Python, Artificial Intelligence, Backtesting. **Keywords:** Educational Trading, Cryptocurrencies, Spring Boot, Python, Artificial Intelligence, Backtesting.

Índice

1. Introducción	5
2. Justificación	5
3. Diseño del Sistema	6
3.1. Diagrama de Clases	6
3.2. Descripción de las Clases del Modelo de Datos	6
3.2.1. Modelo de Dominio (Entities)	7
3.3. Capa de Acceso a Datos (Repositories)	8
3.4. Capa de Servicio (Business Logic Layer)	9
3.4.1. Servicios de Trading y Ejecución	10
3.4.2. Servicios de Gestión de Datos y Contabilidad	10
3.4.3. Servicios de Infraestructura y Usuarios	11
3.5. Capa de Aplicación y Utilidades Transversales	11
3.5.1. Punto de Entrada y Configuración (Main)	12
3.5.2. Interfaz de Usuario (CLI)	12
3.5.3. Gestión de Sesión y Seguridad	12
3.5.4. Utilidades del Sistema	12
3.5.5. Servicios de Gestión de Datos y Contabilidad	13
3.5.6. Servicios de Infraestructura y Usuarios	13
3.6. Capa de Aplicación y Utilidades Transversales	14
3.6.1. Punto de Entrada y Configuración (Main)	14
3.6.2. Interfaz de Usuario (CLI)	14
3.6.3. Gestión de Sesión y Seguridad	14
3.6.4. Utilidades del Sistema	15
3.7. Diagramas de Flujo	15

3.7.1.	Gestión de Usuarios y Acceso	15
3.7.2.	Gestión de Capital (Wallets)	16
3.7.3.	Obtención de Datos (ETL)	17
3.7.4.	Gestión de Estrategias	19
3.7.5.	Motores de Ejecución	20
3.8.	Glosario	22

Referencias Bibliográficas	25
-----------------------------------	-----------

Introducción

El trading automático ha experimentado un auge significativo debido a la volatilidad del mercado de criptomonedas. En este trabajo se analiza la arquitectura inspirada en Freqtrade Team (2026) y el uso de herramientas de modelado como Mermaid Project (2026) y PlantUML Team (2026).

Justificación

La necesidad de este proyecto radica en la dificultad de los inversores humanos para reaccionar en tiempo real ante las fluctuaciones del mercado.

Diseño del Sistema

En este capítulo se detalla la arquitectura técnica del sistema mediante diagramas estructurales y de comportamiento.

Diagrama de Clases

La estructura estática del sistema se ha diseñado siguiendo una arquitectura en capas (Controlador, Servicio, Repositorio y Modelo). La Figura 1 muestra las relaciones entre las entidades principales, destacando el uso de una clase base para la persistencia y la separación de responsabilidades.

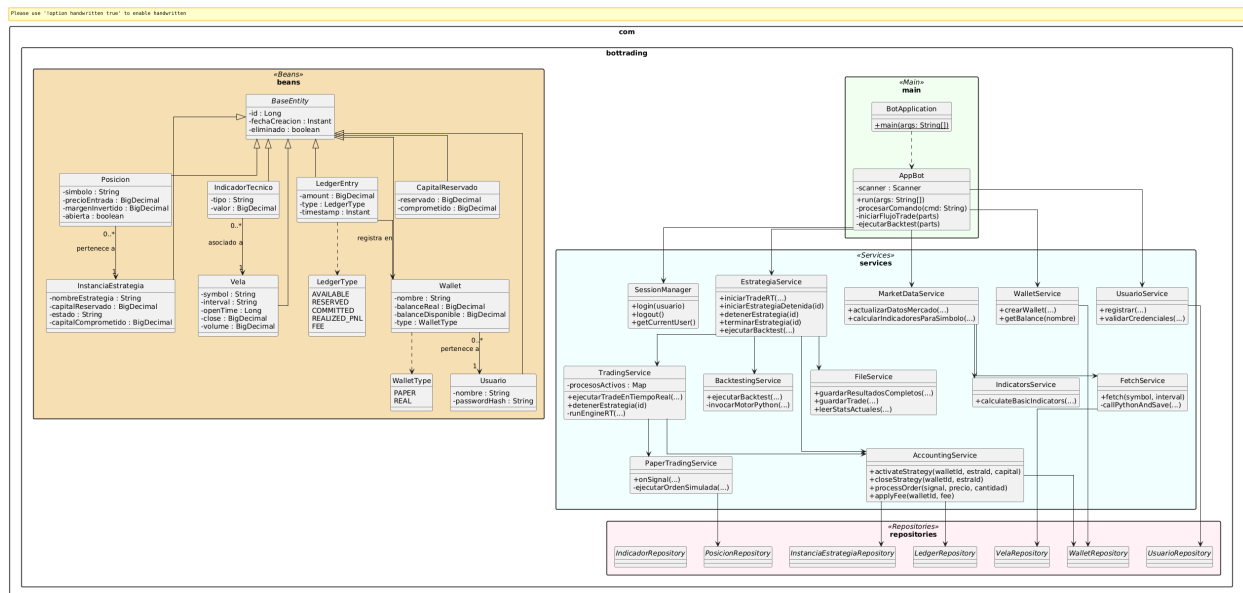


Figura 1: Diagrama de clases detallado de la aplicación

Descripción de las Clases del Modelo de Datos

El diseño del modelo de dominio se basa en entidades JPA que heredan de una clase base común para estandarizar la persistencia. A continuación, se detallan las clases principales que estructuran la lógica de negocio, la gestión de activos y los datos de mercado:

3.2.1. Modelo de Dominio (Entities)

BaseEntity Clase abstracta (@MappedSuperclass) que dota a todas las entidades de un identificador único, marca de tiempo de creación y soporte para borrado lógico.

Usuario Entidad que gestiona la autenticación, almacenando el nombre de usuario y el hash de la contraseña para garantizar la seguridad del acceso.

Wallet Representa la billetera digital del usuario. Implementa un modelo de doble saldo: *balanceReal* (patrimonio total) y *balanceDisponible* (capital libre para nuevas operaciones). Mantiene una relación *Many-to-One* con la entidad *Usuario*.

WalletType Enumerado que define la naturaleza de la billetera. Permite distinguir entre *PAPER* (entorno de simulación con dinero ficticio) y *REAL* (conexión con fondos reales en el exchange).

Vela Entidad fundamental que almacena los datos de mercado en formato OHLCV. Se ha definido una restricción de unicidad compuesta (*UniqueConstraint*) sobre *symbol*, *interval* y *open.time* para garantizar la integridad de las series temporales utilizadas en el backtesting.

VelaDTO Objeto de transferencia diseñado para la ingesta masiva de datos. Utiliza tipos *String* para los valores numéricos, evitando la pérdida de precisión decimal durante la deserialización JSON antes de su conversión a *BigDecimal*.

InstanciaEstrategia Modela la ejecución de una estrategia de trading. Almacena la configuración de riesgo, el capital asignado, los símbolos a operar y el estado actual de la ejecución (activa, detenida, finalizada).

Posicion Representa una operación de mercado individual. Está vinculada a una *InstanciaEstrategia* y almacena el precio de entrada, el margen utilizado y el estado de la operación.

LedgerEntry Registro inmutable de movimientos financieros. Permite auditar el flujo de capital (comisiones, beneficios, reservas) utilizando tipos definidos en *LedgerType*.

CapitalReservado Entidad auxiliar para gestionar el bloqueo de fondos por estrategia, diferenciando entre capital reservado, comprometido en órdenes activas y riesgo latente.

IndicadorTecnico Almacena los valores de análisis técnico (ej. RSI, MACD) asociados a un momento de mercado, permitiendo la trazabilidad de las decisiones del algoritmo.

DTOs Auxiliares (SignalDTO, IndicadorTecnicoDTO) Objetos planos utilizados para la transferencia de datos entre el motor de análisis y la capa de persistencia, desacoplando la lógica de cálculo del almacenamiento.

Capa de Acceso a Datos (Repositories)

La interacción con la base de datos se abstrae mediante el patrón *Repository*, implementado a través de **Spring Data JPA**. Cada interfaz extiende de `JpaRepository`, lo que proporciona operaciones CRUD estándar, además de definir consultas personalizadas mediante *Query Methods* y JPQL (Java Persistence Query Language).

A continuación, se describen los repositorios clave y sus responsabilidades específicas:

InstanciaEstrategiaRepository Gestiona la persistencia de las estrategias. Destaca el método `sumCapitalActivoByWallet`, que realiza una agregación (SUM) para calcular cuánto capital de una billetera está comprometido en estrategias activas, evitando sobregiros. Además, implementa un bloqueo pesimista (`PESSIMISTIC_WRITE`) en `findByIdWithLock` para prevenir condiciones de carrera cuando múltiples hilos intentan modificar el estado de una estrategia simultáneamente.

VelaRepository Optimizado para el manejo de series temporales de datos de mercado. Incluye consultas nativas eficientes para obtener la última vela registrada (`findMaxOpenTime...`), lo cual es crítico para la funcionalidad de descarga incremental de datos (Fetch) sin duplicados. También permite la recuperación ordenada de velas para alimentar los motores de backtesting.

WalletRepository Crítico para la integridad financiera. Utiliza bloqueos pesimistas en las operaciones de lectura (`findByIdWithLock`) para asegurar que ninguna otra transacción modifique el saldo de una billetera mientras se está procesando una orden. Permite consultas filtradas por usuario y tipo de billetera (PAPER/REAL).

LedgerRepository Proporciona la trazabilidad contable del sistema. Permite consultar el historial de transacciones ordenado cronológicamente (`OrderByTimestampDesc`), ya sea filtrando por billetera o por una estrategia específica, facilitando la auditoría de pérdidas y ganancias (PnL).

PosicionRepository Gestiona el estado de las operaciones de mercado. Su método `findByInstanciaAndSin` es fundamental para el motor de trading, ya que permite verificar rápidamente si una estrategia ya tiene una posición abierta en un par de monedas antes de emitir una nueva orden de entrada.

UsuarioRepository Encargado de la gestión de identidades. Abstrae las consultas de autenticación y verificación de existencia de usuarios, asegurando que no se registren nombres duplicados (`existsByNombre`).

IndicadorRepository Permite el almacenamiento y recuperación de análisis técnico calculado previamente. Facilita la limpieza de datos obsoletos mediante operaciones transaccionales de borrado masivo (`deleteByVela`).

Capa de Servicio (Business Logic Layer)

La lógica de negocio de la aplicación se encapsula en servicios de Spring (`@Service`), los cuales orquestan las operaciones entre los controladores, los repositorios y los motores de cálculo externos. A continuación, se detallan los servicios principales:

3.4.1. Servicios de Trading y Ejecución

Estos componentes constituyen el núcleo operativo del sistema, gestionando el ciclo de vida de las estrategias y la comunicación con los procesos de Python.

EstrategiaService Actúa como el orquestador principal del ciclo de vida de las estrategias. Gestiona los comandos de usuario para iniciar (`iniciarTradeRT`), pausar (`detenerEstrategia`) y liquidar (`terminarEstrategia`) las instancias. Coordina la reserva de capital con *AccountingService* antes de delegar la ejecución técnica al *TradingService*.

TradingService Servicio de bajo nivel encargado de la gestión de hilos y procesos del sistema operativo. Utiliza un *ExecutorService* para lanzar y supervisar los scripts de Python (`engine_rt.py`) en hilos independientes. Implementa un mapa concurrente (`ConcurrentHashMap`) para mantener el control de los procesos activos y permitir su detención forzosa en caso de emergencia.

PaperTradingService Simula la ejecución de órdenes en un entorno controlado. Recibe las señales de compra/venta (`SignalDTO`) desde el motor de Python, valida las reglas de gestión de riesgo y, si procede, registra las posiciones simuladas en la base de datos, comunicando los resultados financieros al servicio de contabilidad.

BacktestingService Facilita la validación histórica de estrategias. Prepara los datos de velas almacenados en la base de datos, los transforma a formato JSON y los envía al motor de backtesting (`engine_backtest.py`). Posteriormente, captura los resultados y estadísticas para su almacenamiento.

3.4.2. Servicios de Gestión de Datos y Contabilidad

Garantizan la integridad de la información financiera y la disponibilidad de datos de mercado.

AccountingService Servicio transaccional crítico ('@Transactional') que gestiona el libro mayor (*Ledger*). Es responsable de todas las operaciones monetarias: reservar fondos al activar una

estrategia (`'activateStrategy'`), mover capital a margen comprometido (`'commitCapital'`) y liquidar beneficios o pérdidas (`'closeTrade'`). Implementa bloqueos pesimistas en las billeteras para evitar condiciones de carrera en el saldo.

FetchService Implementa la lógica de sincronización de datos de mercado (ETL). Utiliza un algoritmo de descarga incremental: verifica la última vela registrada en la base de datos y solicita a la API externa únicamente los datos nuevos, optimizando el ancho de banda y el tiempo de procesamiento.

MarketDataService Actúa como una fachada (*Facade*) que simplifica la interacción con los datos de mercado. Coordina la descarga de velas a través de *FetchService* y el cálculo posterior de indicadores técnicos mediante *IndicatorsService*, asegurando que los datos estén actualizados antes de cualquier análisis.

3.4.3. Servicios de Infraestructura y Usuarios

Proporcionan funcionalidades transversales de soporte al sistema.

UsuarioService Gestiona el ciclo de vida de los usuarios. Implementa el registro con hash de contraseñas (utilizando *HashUtils*) y la validación de credenciales. Además, soporta el borrado lógico de cuentas para mantener la integridad referencial de los datos históricos.

FileService Encargado de la persistencia en el sistema de archivos. Genera y actualiza los reportes CSV de operaciones (*trades*) y estadísticas acumuladas (*stats*). Incluye mecanismos de bloqueo para evitar corrupción de archivos durante la escritura concurrente.

Capa de Aplicación y Utilidades Transversales

Para complementar la lógica de negocio, se han implementado componentes de infraestructura que gestionan el ciclo de vida de la aplicación, la interacción con el usuario y la seguridad.

3.5.1. Punto de Entrada y Configuración (Main)

La clase `BotApplication` es el punto de entrada de la aplicación Spring Boot. Implementa una carga personalizada de variables de entorno (utilizando la librería *Dotenv*) para configurar la conexión a base de datos de forma segura sin hardcodear credenciales. Además, silencia los logs internos de Spring y Hibernate para ofrecer una interfaz de línea de comandos (CLI) limpia al usuario.

3.5.2. Interfaz de Usuario (CLI)

La interacción se gestiona mediante la clase `AppBot`, que implementa la interfaz `CommandLineRunner`. Actúa como un bucle de eventos que lee los comandos del usuario (ej: *trade*, *fetch*, *backtest*) y delega la ejecución al servicio correspondiente. Incluye validaciones de sesión mediante `SessionManager` antes de permitir operaciones críticas.

3.5.3. Gestión de Sesión y Seguridad

SessionManager Componente con ámbito de Singleton que mantiene en memoria el estado del usuario actual (`currentUser`). Facilita el acceso global al contexto de seguridad y asegura que, al detener la aplicación (`@PreDestroy`), se cierren ordenadamente todas las estrategias activas.

HashUtils Clase de utilidad estática que encapsula la criptografía de contraseñas utilizando **BCrypt**.

Garantiza que las credenciales de los usuarios se almacenen de forma segura (hasheadas) y nunca en texto plano, cumpliendo con los estándares de seguridad básicos.

3.5.4. Utilidades del Sistema

PathConfig Centraliza la gestión de rutas del sistema de archivos. Resuelve dinámicamente la ubicación de los scripts de Python (*engine_rt.py*, *fetcher.py*) y las carpetas de resultados, independientemente de desde dónde se ejecute el JAR, evitando errores de *.archivo no encontrado*.^{en}

despliegues.

AppConstants Define el protocolo de comunicación entre Java y Python. Contiene las claves de los objetos JSON (ej: `KEY_SYMBOL`, `KEY_PRICE`) y las cabeceras de los archivos CSV, asegurando que ambos lenguajes hablen el mismo idioma sin errores de tipado.

ConsoleLoader Mejora la experiencia de usuario (UX) mostrando animaciones de carga (spinners o puntos) en la consola durante operaciones largas como la descarga de datos o el entrenamiento de modelos, proporcionando feedback visual de que el sistema sigue procesando.

3.5.5. Servicios de Gestión de Datos y Contabilidad

Garantizan la integridad de la información financiera y la disponibilidad de datos de mercado.

AccountingService Servicio transaccional crítico ('@Transactional') que gestiona el libro mayor (*Ledger*). Es responsable de todas las operaciones monetarias: reservar fondos al activar una estrategia ('activateStrategy'), mover capital a margen comprometido ('commitCapital') y liquidar beneficios o pérdidas ('closeTrade'). Implementa bloqueos pesimistas en las billeteras para evitar condiciones de carrera en el saldo.

FetchService Implementa la lógica de sincronización de datos de mercado (ETL). Utiliza un algoritmo de descarga incremental: verifica la última vela registrada en la base de datos y solicita a la API externa únicamente los datos nuevos, optimizando el ancho de banda y el tiempo de procesamiento.

MarketDataService Actúa como una fachada (*Facade*) que simplifica la interacción con los datos de mercado. Coordina la descarga de velas a través de *FetchService* y el cálculo posterior de indicadores técnicos mediante *IndicatorsService*, asegurando que los datos estén actualizados antes de cualquier análisis.

3.5.6. Servicios de Infraestructura y Usuarios

Proporcionan funcionalidades transversales de soporte al sistema.

UsuarioService Gestiona el ciclo de vida de los usuarios. Implementa el registro con hash de contraseñas (utilizando *HashUtils*) y la validación de credenciales. Además, soporta el borrado lógico de cuentas para mantener la integridad referencial de los datos históricos.

FileService Encargado de la persistencia en el sistema de archivos. Genera y actualiza los reportes CSV de operaciones (*trades*) y estadísticas acumuladas (*stats*). Incluye mecanismos de bloqueo para evitar corrupción de archivos durante la escritura concurrente.

Capa de Aplicación y Utilidades Transversales

Para complementar la lógica de negocio, se han implementado componentes de infraestructura que gestionan el ciclo de vida de la aplicación, la interacción con el usuario y la seguridad.

3.6.1. Punto de Entrada y Configuración (Main)

La clase `BotApplication` es el punto de entrada de la aplicación Spring Boot. Implementa una carga personalizada de variables de entorno (utilizando la librería *Dotenv*) para configurar la conexión a base de datos de forma segura sin hardcodear credenciales. Además, silencia los logs internos de Spring y Hibernate para ofrecer una interfaz de línea de comandos (CLI) limpia al usuario.

3.6.2. Interfaz de Usuario (CLI)

La interacción se gestiona mediante la clase `AppBot`, que implementa la interfaz `CommandLineRunner`. Actúa como un bucle de eventos que lee los comandos del usuario (ej: *trade*, *fetch*, *backtest*) y delega la ejecución al servicio correspondiente. Incluye validaciones de sesión mediante `SessionManager` antes de permitir operaciones críticas.

3.6.3. Gestión de Sesión y Seguridad

SessionManager Componente con ámbito de Singleton que mantiene en memoria el estado del usuario actual (`currentUser`). Facilita el acceso global al contexto de seguridad y asegura

que, al detener la aplicación (@PreDestroy), se cierran ordenadamente todas las estrategias activas.

HashUtils Clase de utilidad estática que encapsula la criptografía de contraseñas utilizando **BCrypt**.

Garantiza que las credenciales de los usuarios se almacenen de forma segura (hasheadas) y nunca en texto plano, cumpliendo con los estándares de seguridad básicos.

3.6.4. Utilidades del Sistema

PathConfig Centraliza la gestión de rutas del sistema de archivos. Resuelve dinámicamente la ubicación de los scripts de Python (*engine_rt.py*, *fetcher.py*) y las carpetas de resultados, independientemente de desde dónde se ejecute el JAR, evitando errores de ^{en}archivo no encontrado.^{en} despliegues.

AppConstants Define el protocolo de comunicación entre Java y Python. Contiene las claves de los objetos JSON (ej: `KEY_SYMBOL`, `KEY_PRICE`) y las cabeceras de los archivos CSV, asegurando que ambos lenguajes hablen el mismo idioma sin errores de tipado.

ConsoleLoader Mejora la experiencia de usuario (UX) mostrando animaciones de carga (spinners o puntos) en la consola durante operaciones largas como la descarga de datos o el entrenamiento de modelos, proporcionando feedback visual de que el sistema sigue procesando.

Diagramas de Flujo

A continuación, se describen los procesos principales del sistema, desde la autenticación hasta la ejecución de estrategias en tiempo real.

3.7.1. Gestión de Usuarios y Acceso

El flujo de entrada al sistema valida las credenciales y gestiona la sesión del usuario, permitiendo también el acceso a la ayuda del sistema (ver Figura 2).

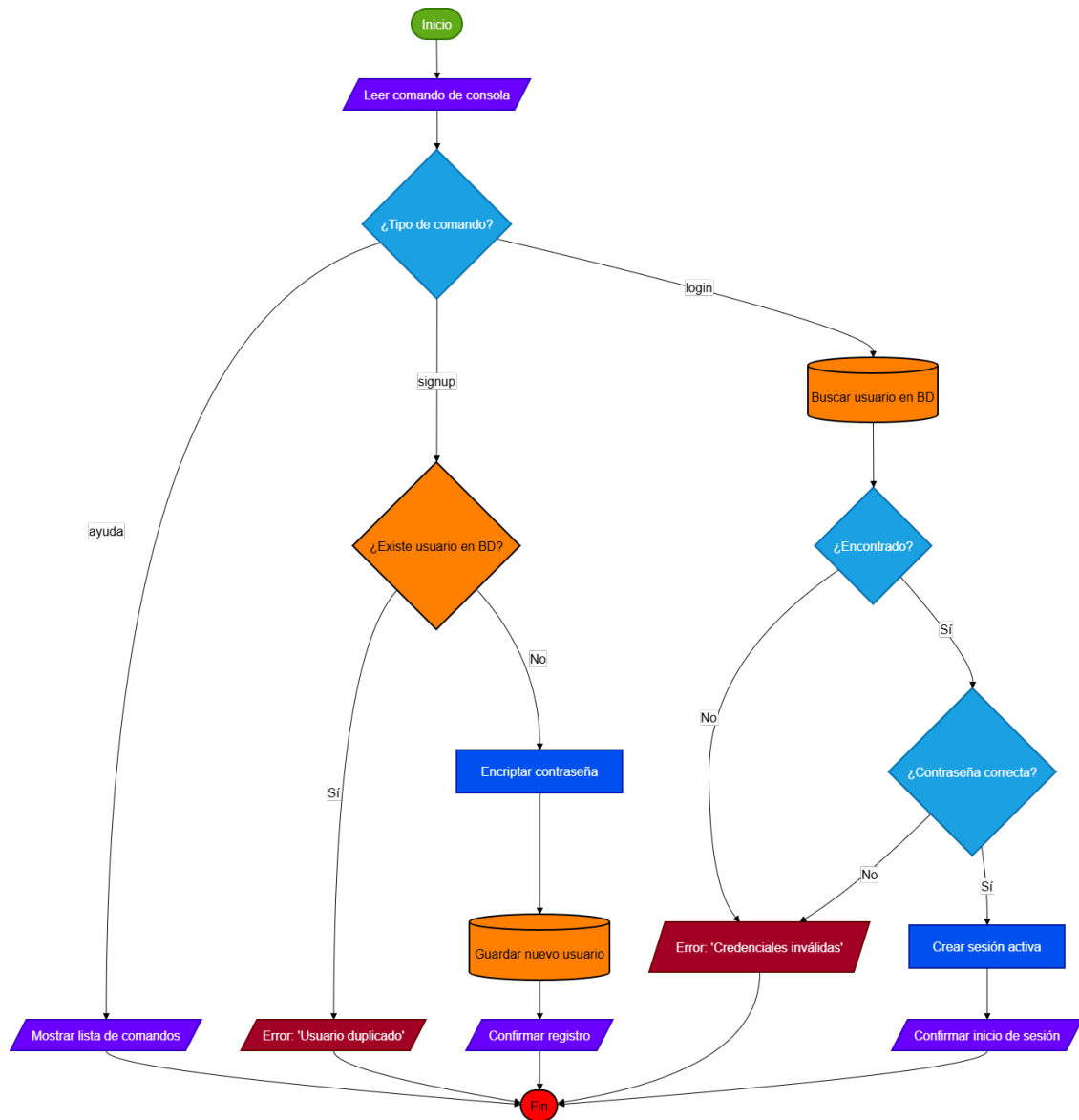


Figura 2: Flujo de autenticación (Login/Signup) y Ayuda

3.7.2. Gestión de Capital (Wallets)

La creación y consulta de billeteras virtuales (Paper Trading) se valida contra la sesión actual del usuario, asegurando la integridad de los datos financieros simulados.

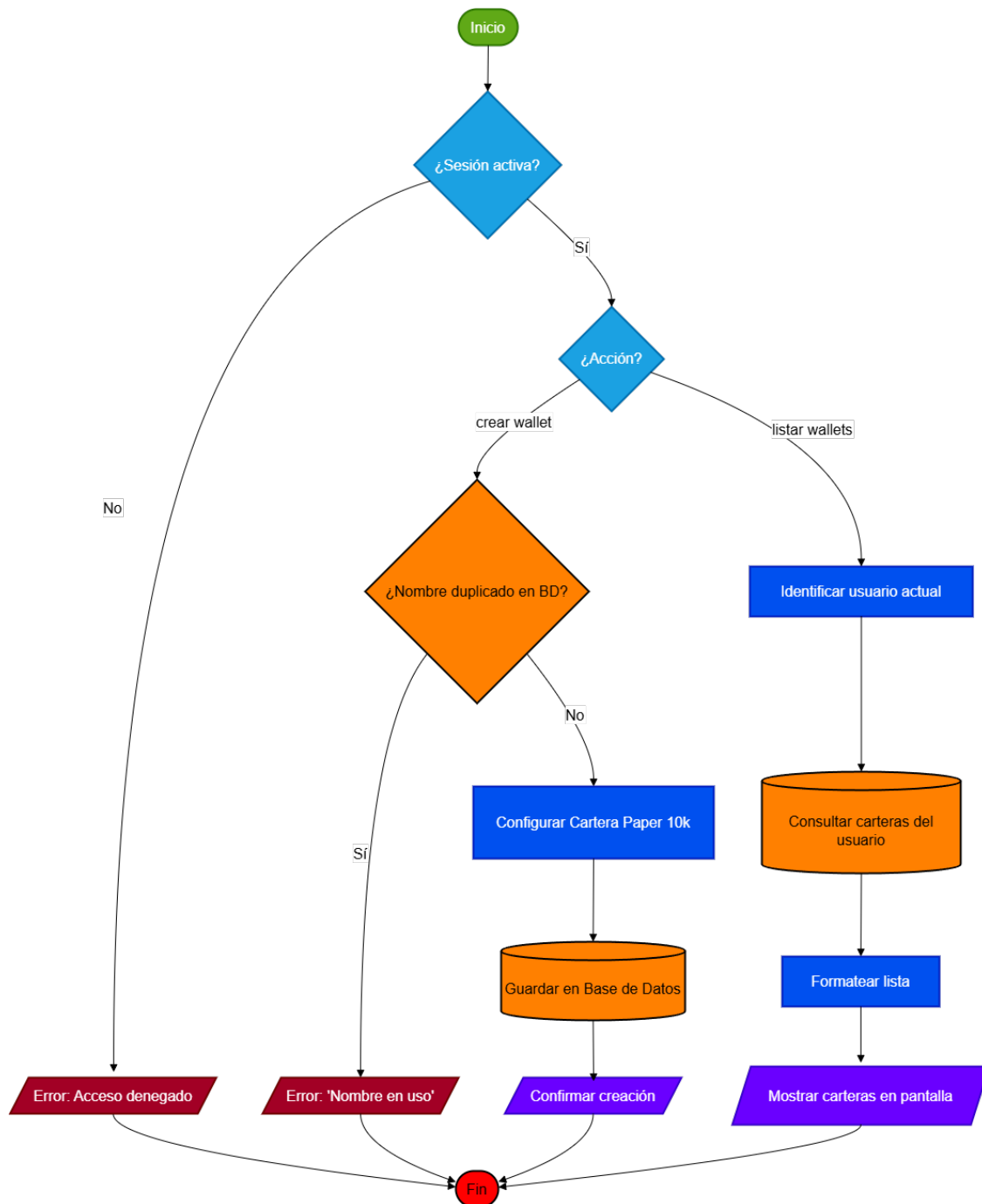


Figura 3: Proceso de creación y listado de Wallets

3.7.3. Obtención de Datos (ETL)

El sistema implementa un mecanismo de descarga incremental para optimizar las peticiones a la API de Binance, tal y como se ilustra en la Figura 4.

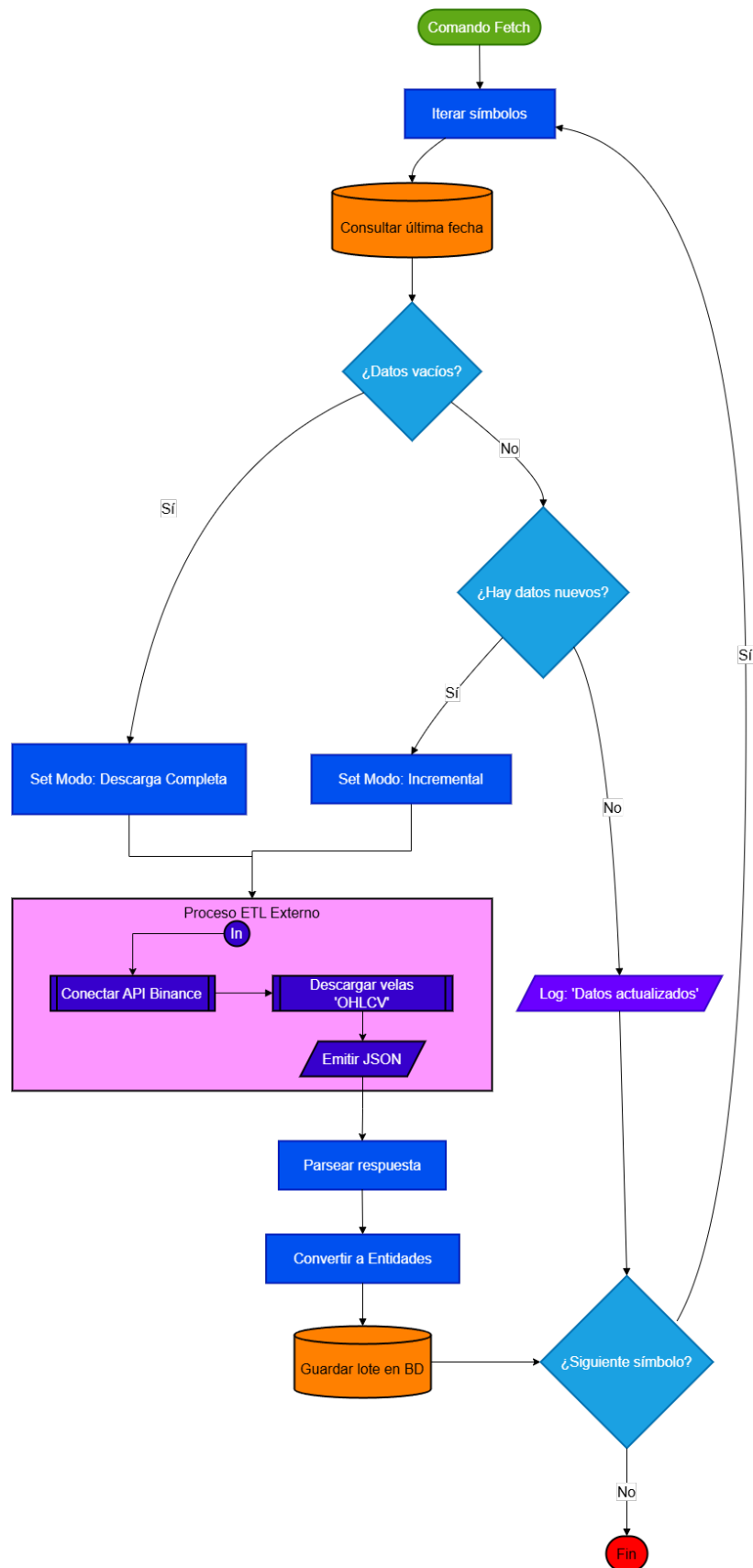


Figura 4: Flujo de obtención y sincronización de datos de mercado (Fetch)

3.7.4. Gestión de Estrategias

El ciclo de vida de las estrategias permite al usuario listar los scripts disponibles en disco y consultar el estado de las instancias en base de datos.

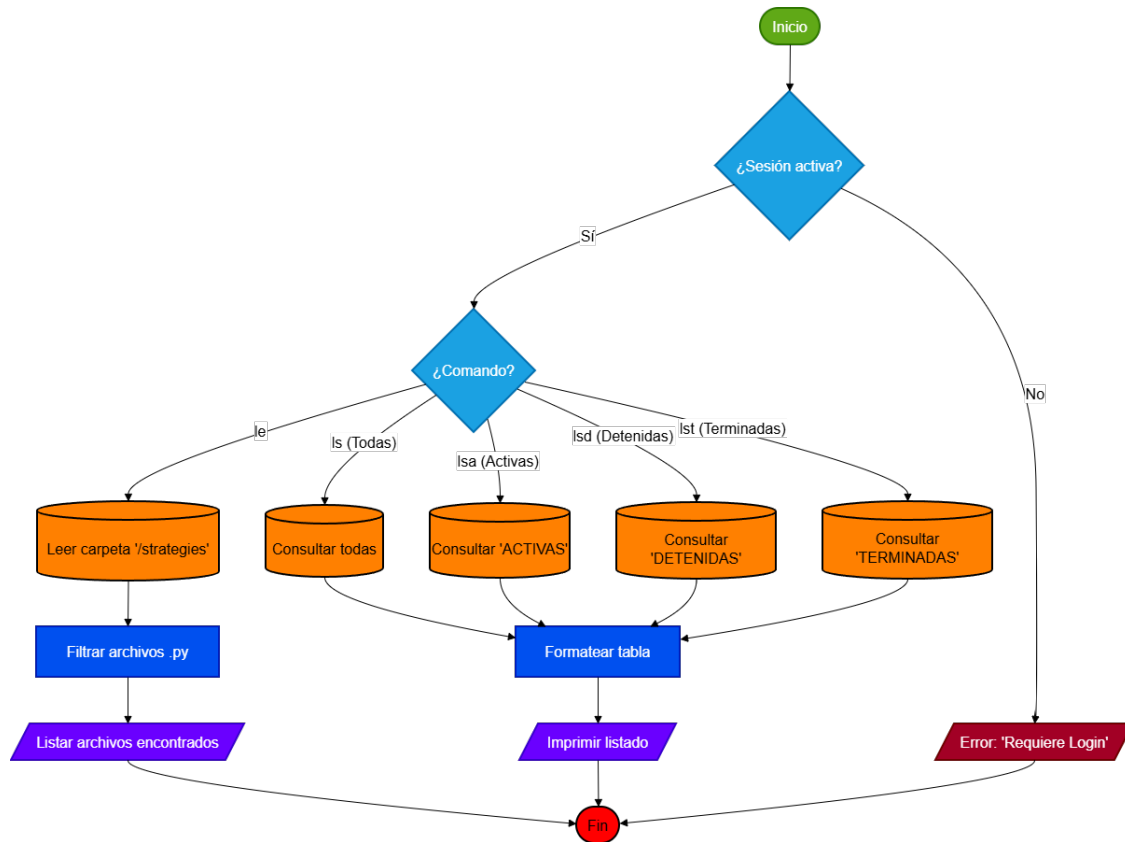


Figura 5: Flujo de comandos de listado (ls, le, lsa)

La Figura 6 detalla las operaciones de control de ciclo de vida: pausar (*stop*), reanudar (*start*) y liquidar (*term*) una estrategia.

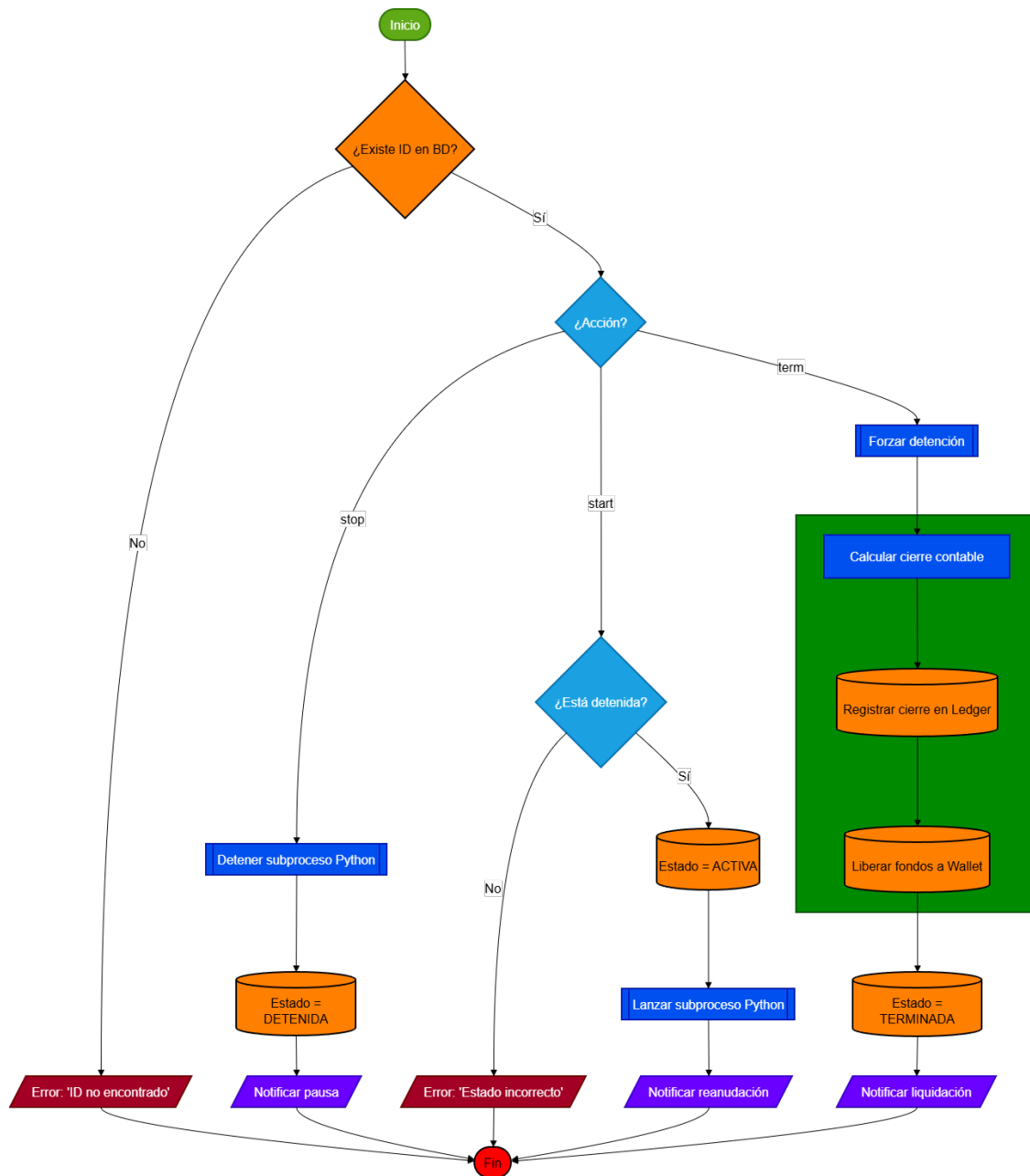


Figura 6: Ciclo de vida: Start, Stop y Terminate

3.7.5. Motores de Ejecución

El núcleo del sistema reside en los motores de simulación y ejecución real. La Figura 7 muestra el flujo de validación histórica.

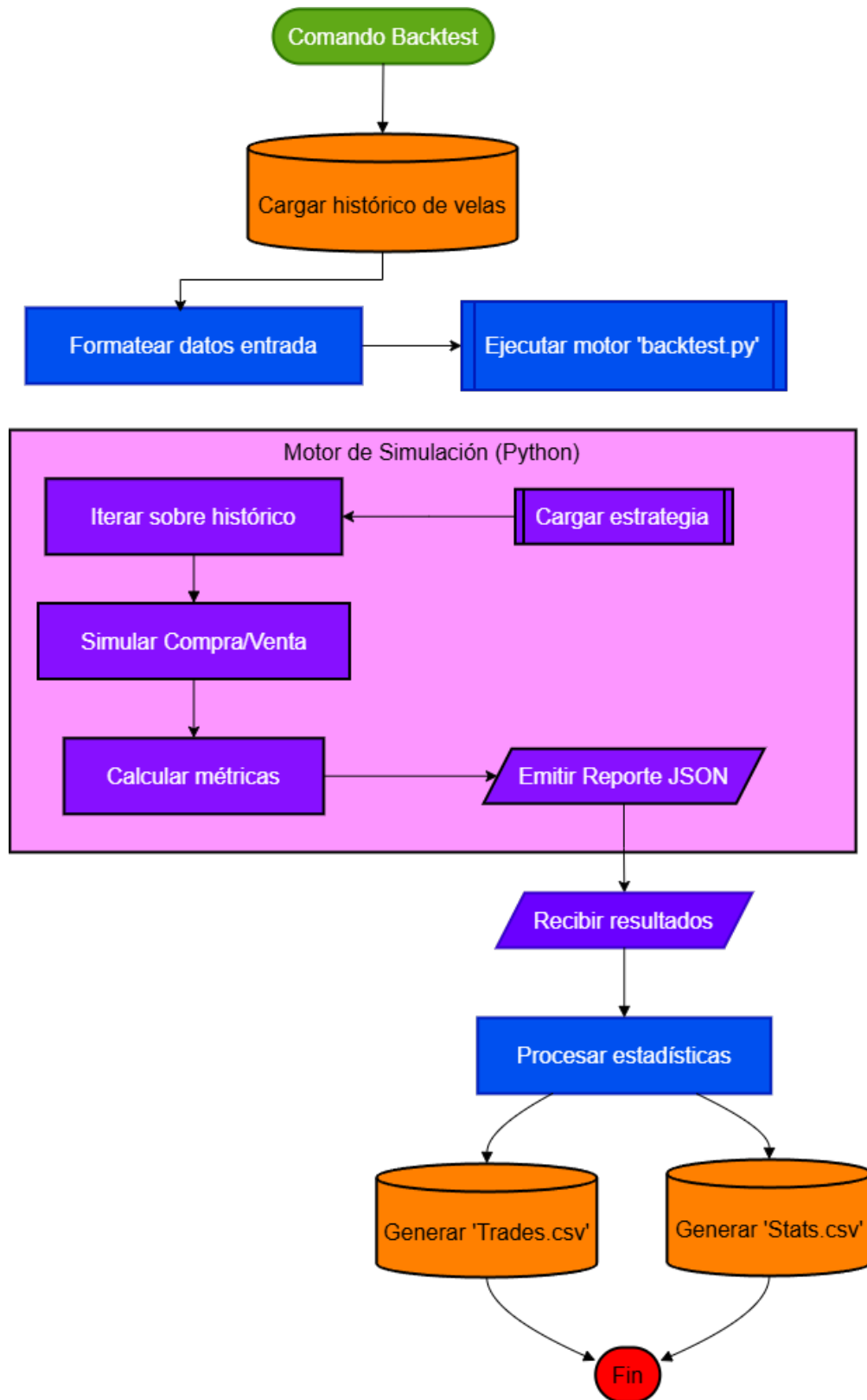


Figura 7: Flujo de ejecución del motor de Backtesting

Finalmente, la Figura 8 ilustra la arquitectura híbrida Java-Python para la operativa en tiempo real, incluyendo la gestión transaccional del Ledger.

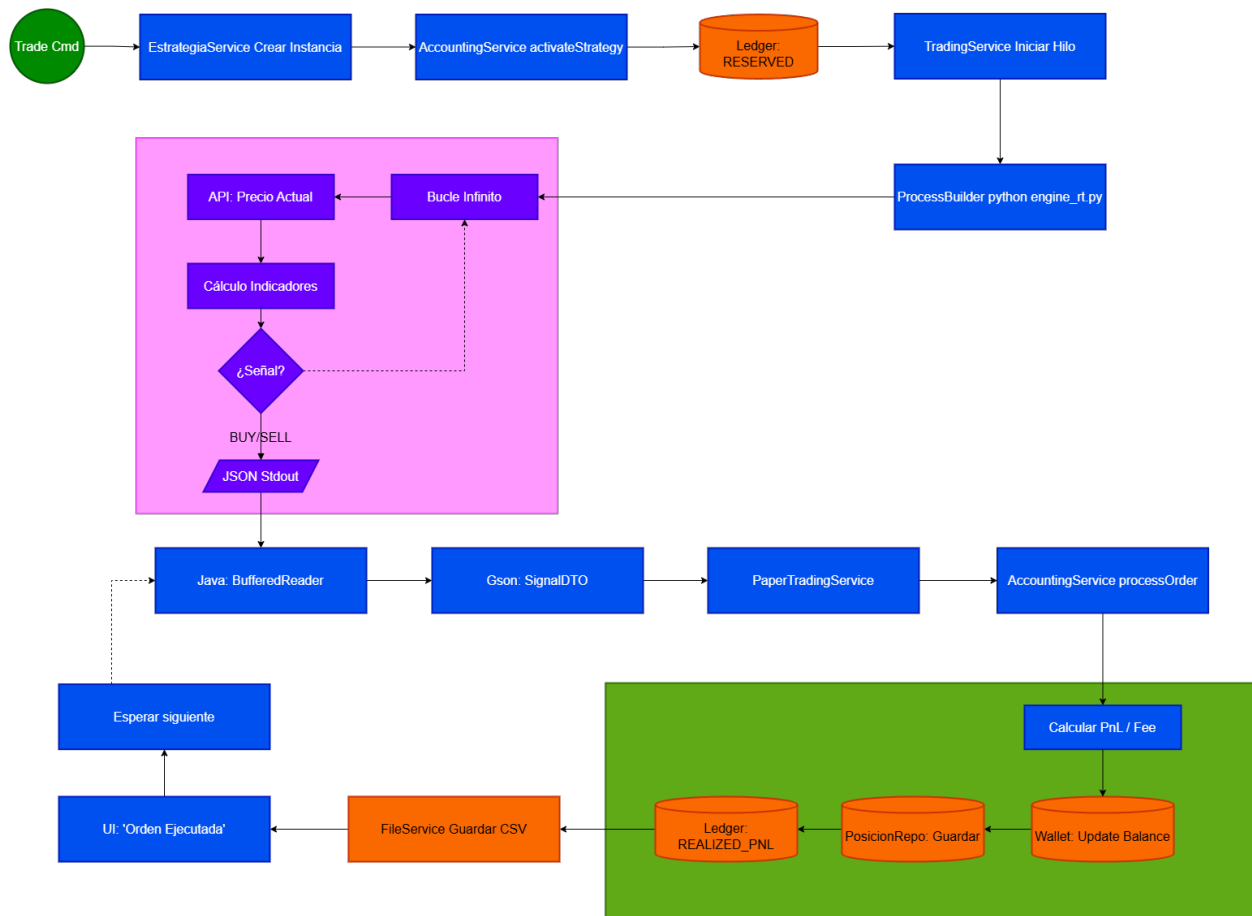


Figura 8: Flujo de Trading en Tiempo Real y comunicación Java-Python

Glosario

Para la correcta comprensión del dominio del problema, se definen a continuación los conceptos económicos y técnicos fundamentales utilizados en el diseño del sistema:

Exchange (Casa de Intercambio) Plataforma digital que facilita la compraventa de activos financieros, como criptomonedas, actuando como intermediario y proveedor de liquidez y datos de mercado. En este proyecto, se simula la interacción con un Exchange (ej. Binance) para la obtención de datos históricos y precios en tiempo real.

OHLCV Acrónimo de *Open, High, Low, Close, Volume*. Es el formato estándar para representar la acción del precio de un activo financiero durante un intervalo de tiempo específico (Vela o *Candlestick*).

- **Open:** Precio al inicio del intervalo.
- **High:** Precio máximo alcanzado.
- **Low:** Precio mínimo alcanzado.
- **Close:** Precio al cierre del intervalo.
- **Volume:** Cantidad total del activo negociada.

En el sistema, esta estructura se modela mediante la entidad `Vela`.

Wallet (Billetera Digital) Entidad encargada de custodiar y gestionar el capital del usuario. En el contexto de la aplicación, se implementa un sistema de *doble saldo* para distinguir entre el patrimonio total (*Balance Real*) y el capital libre para operar (*Balance Disponible*).

Ledger (Libro Mayor) Registro contable inmutable y secuencial donde se anotan todas las transacciones financieras. Su función es garantizar la integridad de los datos y permitir la auditoría de movimientos como reservas de fondos, comisiones o beneficios realizados. En el sistema, cada movimiento genera una entrada de tipo `LedgerEntry`.

Estrategia de Trading Conjunto de reglas lógicas y matemáticas que determinan de forma autónoma cuándo comprar o vender un activo. Estas reglas se definen en scripts de Python y son orquestadas por la entidad `InstanciaEstrategia` en Java.

Indicador Técnico Cálculo matemático derivado del precio y/o volumen histórico de un activo, utilizado para predecir movimientos futuros del mercado.

- **RSI (Relative Strength Index):** Oscilador de momento que mide la velocidad y el cambio de los movimientos de precios para identificar condiciones de sobrecompra o sobreventa.

- **MACD (Moving Average Convergence Divergence):** Indicador de seguimiento de tendencia que muestra la relación entre dos medias móviles del precio de un activo.

El sistema persiste estos valores en la entidad `IndicadorTecnico` para su análisis posterior.

Paper Trading Modalidad de simulación bursátil que permite operar con dinero ficticio en un entorno de mercado real. El sistema soporta este modo mediante el servicio `PaperTradingService`, permitiendo validar estrategias sin riesgo financiero.

PnL (Profit and Loss) Término que hace referencia a las ganancias o pérdidas netas obtenidas en una operación o en un periodo determinado. Se calcula como la diferencia entre el valor de salida y el valor de entrada de una posición, descontando las comisiones.

Referencias Bibliográficas

- Freqtrade Team. (2026). *Freqtrade Documentation: Strategy Customization*. Consultado el 18 de febrero de 2026, desde <https://www.freqtrade.io/en/stable/>
- Mermaid Project. (2026). *Mermaid Live Editor: Diagramming and charting tool*. Consultado el 18 de febrero de 2026, desde <https://mermaid.live/>
- PlantUML Team. (2026). *PlantUML: Open-source tool to create diagrams from plain text*. Consultado el 18 de febrero de 2026, desde <https://www.plantuml.com/>